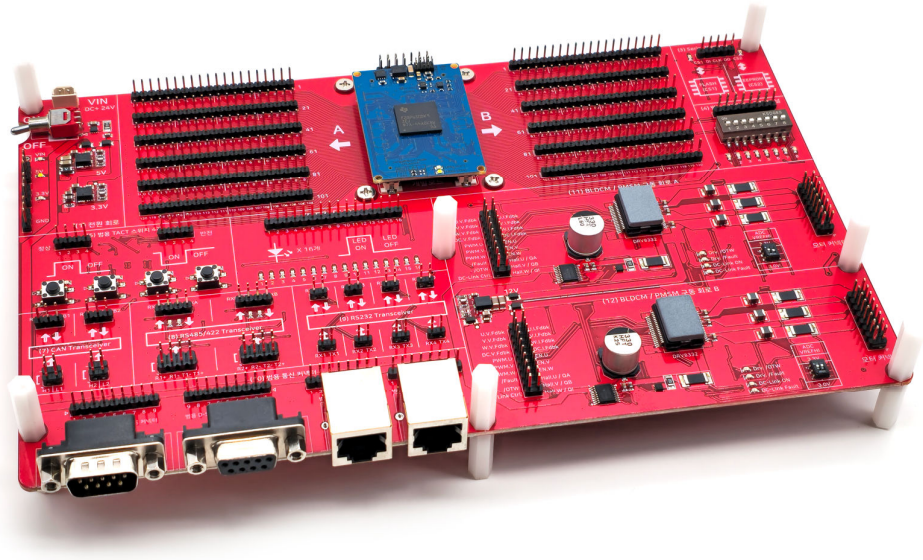
* 본 자료는 모터구동 학습용 예제로만 사용하십시오. 실제 시스템 적용으로 인한 문제에 대해서는 싱크웍스에서 책임지지 않습니다. 또한, 기술적인 지원도 할 수 없음을 알려드립니다.

* 본 예제 프로그램은 Texas Instruments(TI)사에서 TMS320F28X 프로세서 사용자들을 대상으로 공개한 Digital Motor Control Library를 기초로 만들어졌으며, SMC300 개발보드에 적합하도록 일부 수정되었습니다. (참조자료 : TI ControlSUITE에 포함된 DMC library)

* Revision 2026. 05. 21. (v1.21)

Incremental Encoder를 이용한 3상 PMSM 제어 (FOC) (SMC300 개발보드용)

F28P65x
모듈 탑재
기준 안내



* Revision History

날 짜	내 용
25. 06. 16.	- V1.0 / 초판
25. 12. 23.	- V1.1 / SMC300 EVM V110 회로 설계에 맞춰 일부 수정
26. 05. 20.	- V1.2 / F28P559SJ9-Q1 모듈 사용에 맞춰 내용 수정
26. 05. 21.	- V1.21 / F28P65x 모듈 사용에 맞춰 내용 수정

* SMC300 개발보드 V100 → V110 회로설계 주요 변경 부분
  - 모터 구동 부, DC-LINK 전원 ON/OFF용 물리 스위치(Toggle) 제거
    (eFuse 소자에 의한 전자제어로 DC-LINK 전원 ON/OFF 제어 가능)

목차

1 실습 준비 4

2 주의 사항 4

3 실습 예제 설치 4

4 예제 실행 5

4.1 예제 실행 준비하기 7

4.2 실습 따라 하기 (BUILD LEVEL 1) 13

4.3 실습 따라 하기 (BUILD LEVEL 2) 22

4.4 실습 따라 하기 (BUILD LEVEL 3) 25

4.5 실습 따라 하기 (BUILD LEVEL 4) 29

4.6 실습 따라 하기 (BUILD LEVEL 5) 32

1 실습 준비

본 예제는 아래의 개발환경에서 테스트되었습니다.

>> **CCS 버전** : v12.8.1.00005

>> **Compiler 버전** : TI v25.11.0.LTS

>> **개발보드** : SMC300 모터제어 개발보드

>> **TMS320F28X 모듈** : TMS320F28P650DK9 모듈 / TMS320F28P659DK8-Q1 모듈

2 주의 사항

전력 변환이나 모터 구동 등의 실습은 고-전압 혹은 고-전류에 노출되어 있기 쉽습니다. 이는 부주의로 인한 사고 발생 시, 장비들을 손상시킬 뿐만 아니라 개발자들께도 위험을 줄 수 있습니다. 따라서 실습을 진행함에 있어 명시된 절차를 필히 지켜 주셔야 합니다. 또한 소비전류 수치의 확인 및 제한이 용이한 DC 전원공급기의 사용을 권고 드리며, 안전한 실습을 위해 예제 프로그램의 실행을 멈추기 전, 반드시 DC-LINK에 공급된 전원을 차단해주십시오. (SMC300 개발보드는 eFuse 스위치의 전자제어를 통해 컨트롤 전원과 별개로 모터에 공급되는 DC-LINK 전원을 차단할 수 있습니다. / SMC300 개발보드 매뉴얼을 참조하세요.)

3 실습 예제 설치

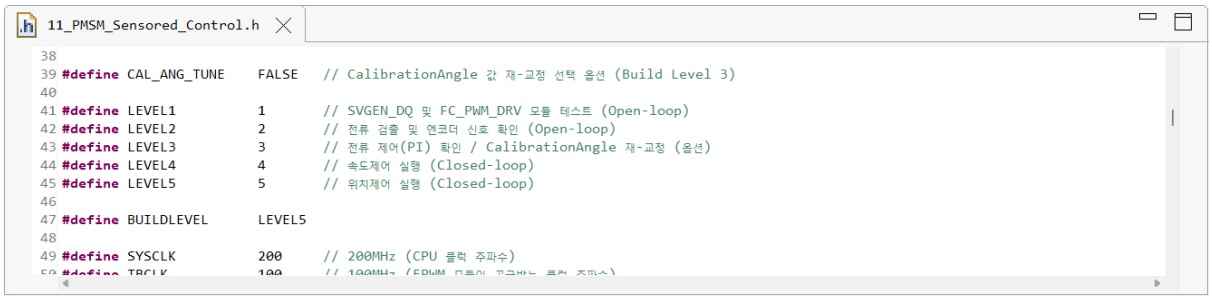
싱크웍스 홈페이지를 통해 다운로드 하신 예제는 실행형 설치 패키지로 제작되어 있으며, 압축해제 후 실행해주시면 지정된 경로에 자동으로 설치됩니다. (설치 파일 실행에 문제가 있다면, 관리자 권한으로 실행해보세요.)

예제의 설치 경로는 아래와 같습니다.

>> **C:\SyncWorks\SMC300_EVM_28P65x\c28x**

4 예제 실행

SMC300 개발보드용 'PMSM_Sensored_Control' 예제는 헤더파일의 전-처리 정의구문을 통해 서로 다른 5개 프로그램이 선택되도록 되어 있습니다. 예제에 포함된 헤더파일에 정의된 빌드 레벨 옵션(BUILDLEVEL)을 원하는 값으로 설정하면 그에 맞는 코드들이 선택적으로 컴파일 됩니다.



[Build Level 1]

Open-Loop 구동 단계로, 프로그램이 임의로 가공한 회전자(Rotor) 위치정보(rg1)를 통해 모터를 회전시킵니다. Space-Vector PWM 생성을 담당하는 SVGEN_DQ, FC_PWM_DRV 모듈의 정상동작 여부를 확인할 수 있습니다.

[Build Level 2]

Open-Loop 구동 단계로, 임의의 회전자 위치정보(rg1)로 모터를 회전시키면서 칩의 On-Chip ADC를 통해 모터의 상-전류 정보를 얻어내고 이를 확인합니다. 또한, Encoder가 출력하는 신호를 해석하여 실제 모터의 회전자 위치 정보를 파악하고 Encoder와 EQEP 모듈 간의 연결 상태, EQEP 모듈의 정상동작 여부를 확인합니다.

[Build Level 3]

Current Closed-Loop 단계로, On-Chip ADC를 통해 얻어낸 상-전류에 PI 제어기를 연결하여 전류(토크)를 일정하게 제어하여 모터를 회전시키며, Encoder로부터의 위치정보는 실제 모터구동에 이용하지 않고 앞서의 Build Level 1, 2와 마찬가지로 회전자 위치에 대해 Open-Loop로 모터를 구동합니다. Build Level 3부터 5까지는 모터 구동 전 Encoder의 Index 신호를 찾아 모터 회전자의 초기위치를 잡는 PrePosInitialize( ) 함수가 실행됩니다.

(옵션) 'PMSM_Sensored_Control.h' 헤더파일의 BUILDLEVEL 설정을 'LEVEL3'으로 정의하고, 추가로 헤더파일의 'CAL_ANG_TUNE' 설정을 'TRUE' 로 정의할 경우, Encoder 신호를 입력 받아 처리하는 QEP_DRV S/W 모듈의 CalibrationAngle 값 교정을 병행할 수 있습니다. (자세한 내용은 이어지는 Level 3 실습 안내 절을 참고하세요.)

[Build Level 4]

Speed Closed-Loop 단계로, 앞서의 Level 1~3 단계에서 TMS320F28X 칩의 On-Chip ADC, Enhanced QEP Module 등을 통해 확인하고 검증한 상-전류, 엔코더로부터의 회전자 위치정보 등을 기초로 PMSM을 제어하는 단계입니다. Q축 전류 PI 제어 Loop 외곽에 속도제어용 PI 제어모듈을 추가하여 부하의 변동에도 사용자의 속도지령을 추종하도록 합니다. Build Level 3부터 5까지는 모터 구동 전 Encoder의 Index 신호를 찾아 모터 회전자의 초기위치를 잡는 루틴이 실행됩니다. - PrePosInitialize( )

[Build Level 5]

Position Closed-Loop 단계로, Build Level 4와 유사하지만, Q축 전류 PI 제어 Loop 외곽에 속도 PI 제어 모듈 대신 위치제어용 PI 제어 모듈을 연결하여 부하의 변화에도 모터의 회전자가 사용자의 위치지령을 추종하도록 합니다. Build Level 3부터 5까지는 모터 구동 전 Encoder의 Index 신호를 찾아 모터 회전자의 초기위치를 잡는 PrePosInitialize( ) 함수가 실행됩니다.

이어지는 예제 따라 하기 안내 과정은 Code Composer Studio v12.x 와 TMS320F28P65x 모듈, SDS200i 절연 JTAG 에뮬레이터 사용을 전제로 작성되었습니다.

4.1 예제 실행 준비하기

1. 아래의 경로에서 Code Composer Studio(CCS)를 다운로드 한 후 PC에 설치합니다.

>> https://www.ti.com/tool/download/CCSTUDIO

2. Code Composer Studio를 실행합니다.

바탕화면 단축아이콘이나 프로그램 메뉴를 통해, PC에 설치된 CCS를 실행합니다.

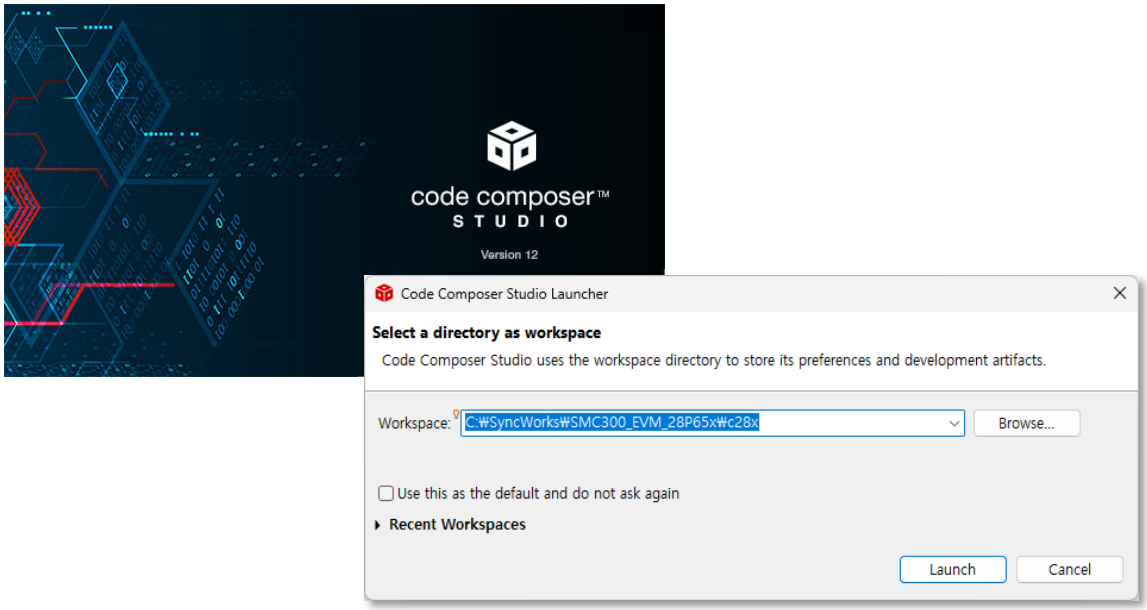
본 예제 프로그램은 CCS v12.8.1.00005 버전에서
제작 되었습니다. (컴파일러 : TI v25.11.0.LTS)

3. Workspace 경로를 선택합니다.

CCS Splash 이미지가 화면에 잠시 보이고, Workspace Launcher 창이 나타나게 됩니다. 이 창에서 Workspace를 예제의 설치경로 보다 한 단계 위 상위 경로로 지정합니다.

(Workspace 경로는 항상 예제 설치 경로보다 상위의 경로로 설정합니다. / 창 하단에 위치한 'Use this as default and do not ask again' 옵션은 선택하지 않습니다.)

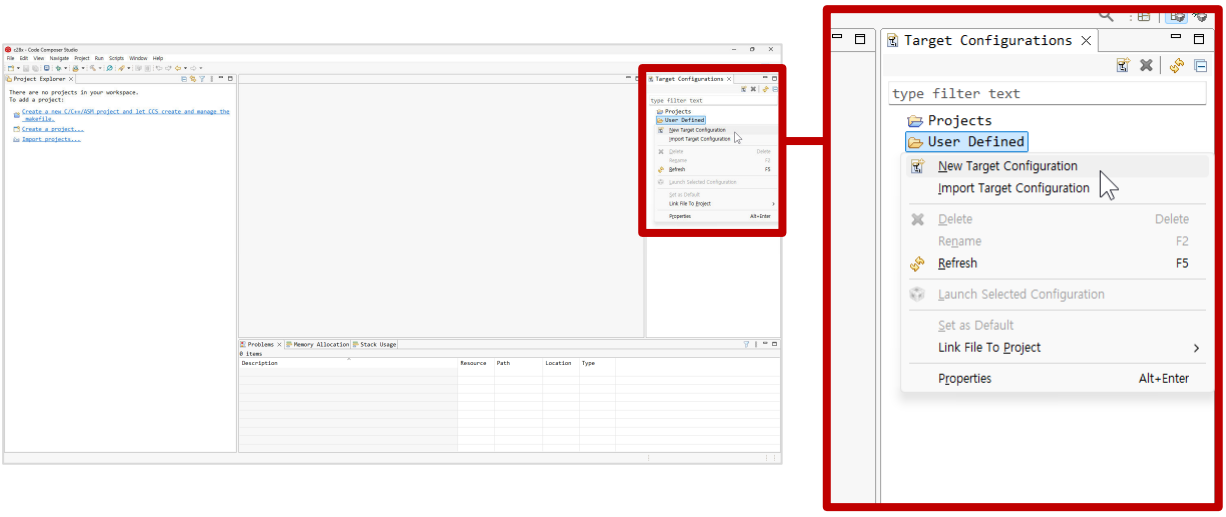
>> 예제 경로 : C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_PMSM_Sensored_Control
>> Workspace : **C:\SyncWorks\SMC300_EVM_28P65x\c28x**



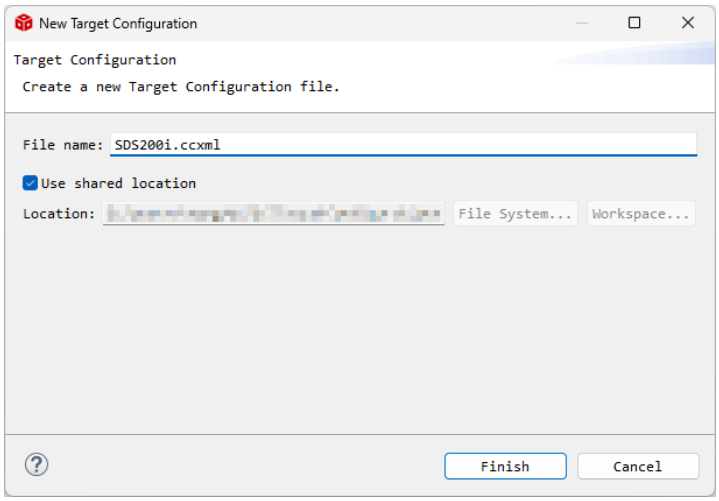
4. JTAG 에뮬레이터 종류 및 디버깅 대상(Target) 프로세서를 설정 합니다.

CCS 상단메뉴 View → Target Configuration을 선택합니다.

화면에 나타난 'Target Configurations' 창에서 'User Defined' 항목을 마우스 우측버튼으로 클릭합니다. 단축메뉴가 나타나면 'New Target Configuration' 항목을 클릭합니다. (미리 저장해둔 Target Configuration 설정이 있을 경우 이를 그대로 사용하셔도 무방합니다. 단, 대상 프로세서는 모듈에 따라, 'TMS320F28P650DK9 혹은 TMS320F28P659DK8-Q1' 로 설정해야 합니다.)

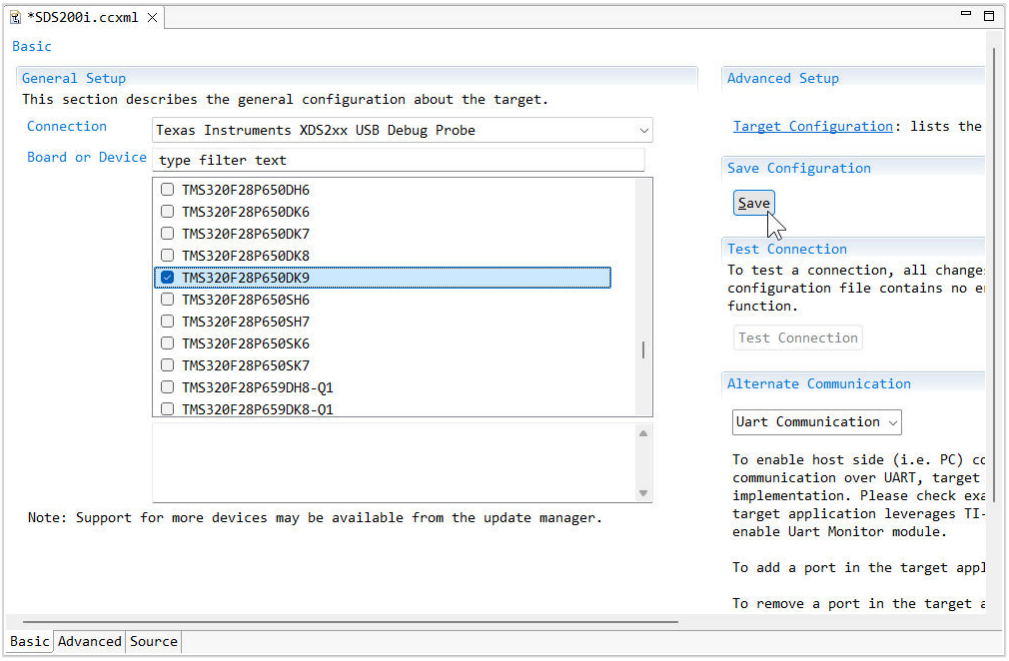


화면에 나타난 'New Target Configuration' 창의 'File name: ' 란에 적당한 이름을 기입합니다. 확장자인 *.ccxml 만 그대로 유지한다면 어떤 이름도 무방합니다. 구분하기 좋은 이름으로 새 타겟환경설정 파일을 생성하세요. 이름을 기입한 후 'Finish' 버튼을 클릭합니다.

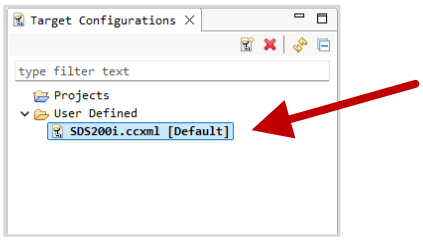


화면 중앙에 생성된 'SDS200i.ccxml' 창(여러분은 전 단계에서 설정하신 이름의 창이 나타날 것 입니다.)에서 'Connection', 'Device' 항목들을 설정합니다. 설정이 끝나면 창 우측의 'Save' 버튼을 클릭하여 변경한 내용을 저장합니다.

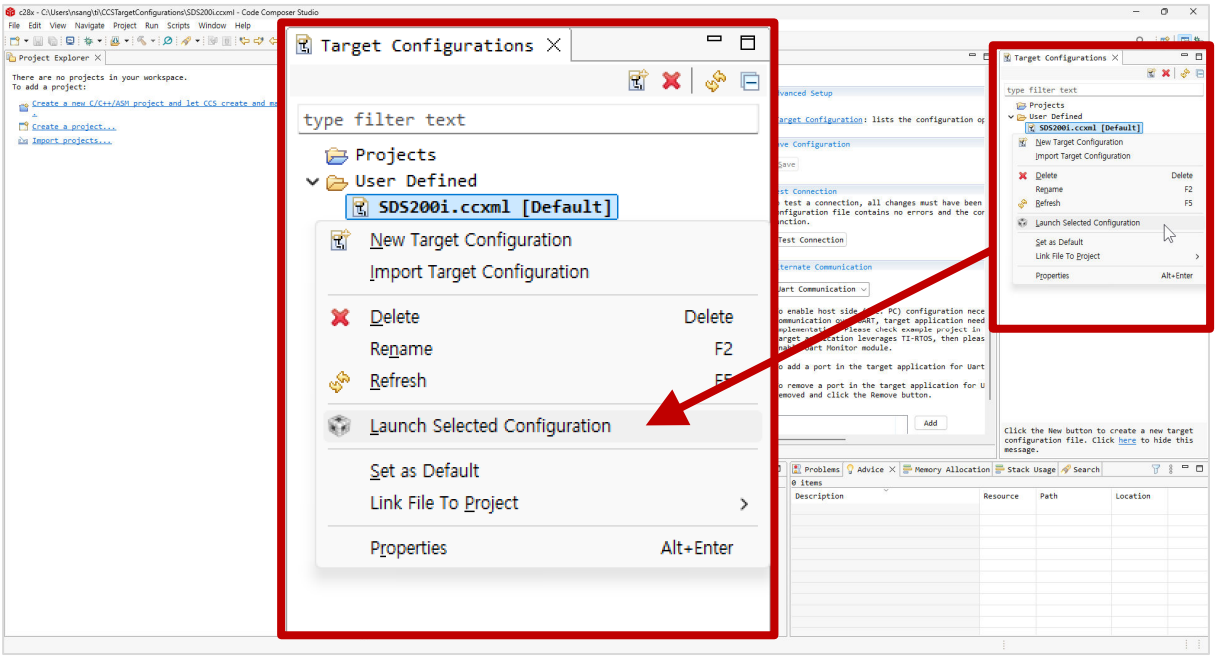
(Device 항목은 모듈에 따라 'TMS320F28P650DK9 혹은 TMS320F28P659DK8-Q1'을 선택하시고, Connection 항목은 사용하시는 JTAG 에뮬레이터에 따라 적합한 드라이버를 선택하시면 됩니다.)

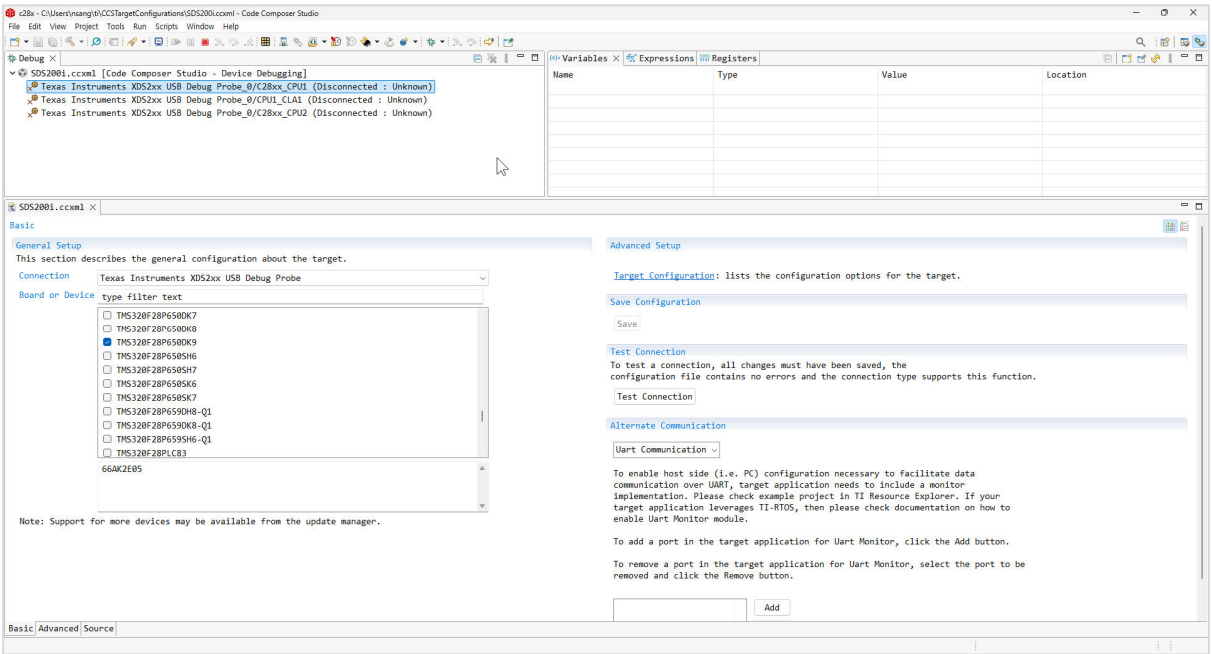


설정 저장을 완료하면 아래의 그림과 같이 CCS 화면 우측에 띄워둔 Target Configurations 창에서 앞서 설정한 타겟 환경설정 파일 'SDS200i.ccxml'를 확인할 수 있습니다.



Target Configurations 창에서 해당 환경설정 파일을 마우스 우측버튼으로 클릭하고 단축메뉴가 나타나면 'Launch Selected Configuration' 항목을 선택합니다. 그렇게 하면 자동으로 CCS Debug 화면구성(Perspective)으로 전환되게 됩니다. (CCS Edit → CCS Debug)


Target Configurations
type filter text
Projects
User Defined
SDS200i.ccxml [Default]
New Target Configuration
Import Target Configuration
Delete
Rename
Refresh
Launch Selected Configuration
Set as Default
Link File To Project
Properties
Alt+Enter


Debug
Variables
Expressions
Registers
SDS200i.ccxml
Basic
General Setup
Advanced Setup
Save Configuration
Test Connection
Alternate Communication
Add

5. SMC300 개발보드에 전원을 공급하고, JTAG 에뮬레이터를 Reset 합니다.

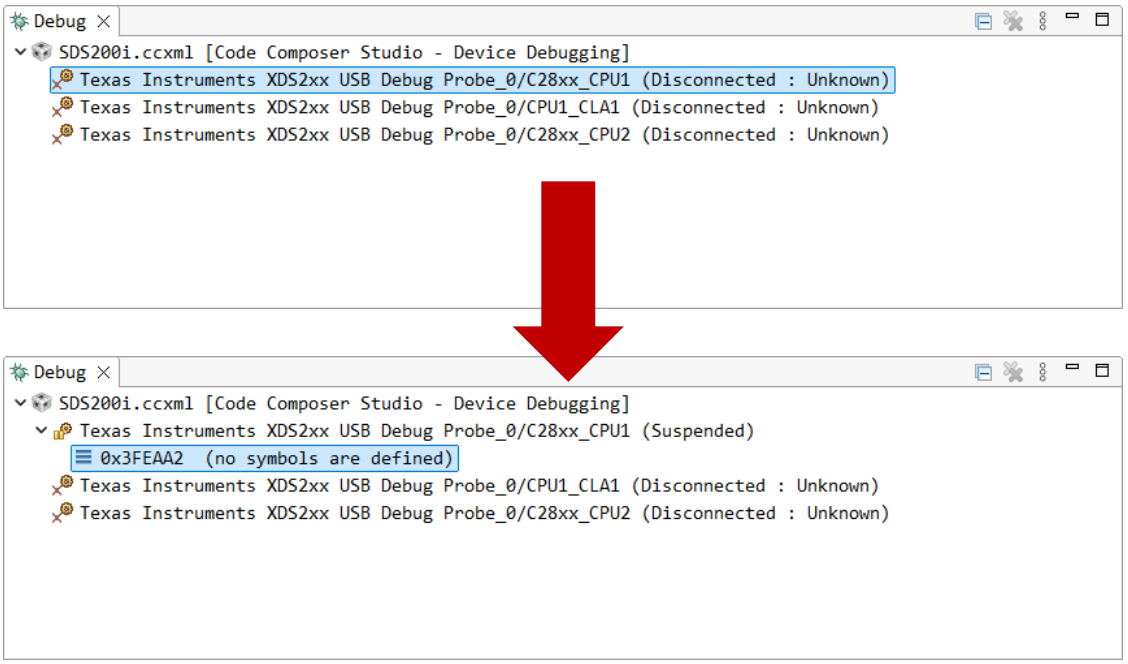
TMS320F28P65x 모듈이 탑재된 SMC300 개발보드에 DC +24V의 전원을 공급하고, PC와 연결된 JTAG 에뮬레이터의 JTAG 커넥터를 개발보드의 JTAG 연결용 핀-헤더(2.54mm 2x7 Male)에 연결합니다. (관련 세부 내용은 매뉴얼 참조)

SMC300 개발보드의 전원 스위치(좌측상단)를 'ON' 위치로 조작하여 보드에 전원을 공급합니다. 단, (SMC300 개발보드의 각종 인터페이스 위치와 조작에 관련한 세부내용은 제품 매뉴얼을 참조하세요.)

CCS 상단메뉴 Run → Reset → Reset Emulator를 선택하여 JTAG 에뮬레이터를 Reset 합니다. 메뉴 항목 클릭 후 화면에 아무런 메시지도 나타나지 않고 변화가 없으면 정상입니다. PC와 JTAG 에뮬레이터 간의 연결에 문제가 있거나, SMC300 개발보드 및 모듈에 전원이 공급되지 않은 경우 CCS 화면 하단의 콘솔(Console) 창에 오류 메시지가 나타납니다.

6. 프로세서와 CCS를 연결(Connect) 합니다.

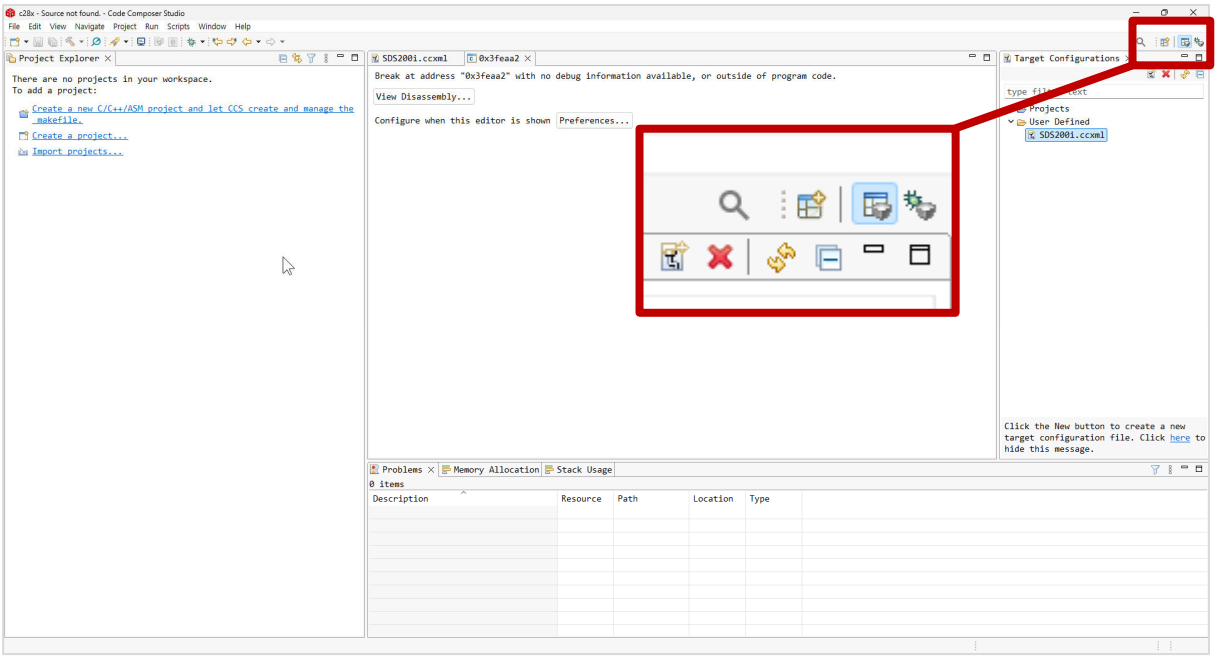
CCS 상단메뉴 Run → Connect Target을 클릭하여 CCS와 TMS320F28P65x 칩의 CPU1을 연결합니다. 정상 연결되면 CCS 화면 좌측 상단의 Debug 창이 아래와 같이 변경됩니다.



7. CCS Edit 화면구성(Perspective)으로 전환하여, 예제 실습을 준비 합니다.

CCS 상단메뉴 Window → Perspective → Open Perspective → CCS Edit 를 선택하거나, CCS 화면 우측 상단의 'CCS Edit' 아이콘을 클릭하여 CCS Edit 화면구성(Perspective)으로 전환합니다. 이제 예제 프로젝트를 불러와서 실습을 진행할 준비가 완료 되었습니다.

위 과정이 정상 진행되면 현재 PC 화면은 아래와 같을 것 입니다. 그렇지 않다면 앞서의 과정 중 빼먹은 것은 없는지 잘 살펴보세요.

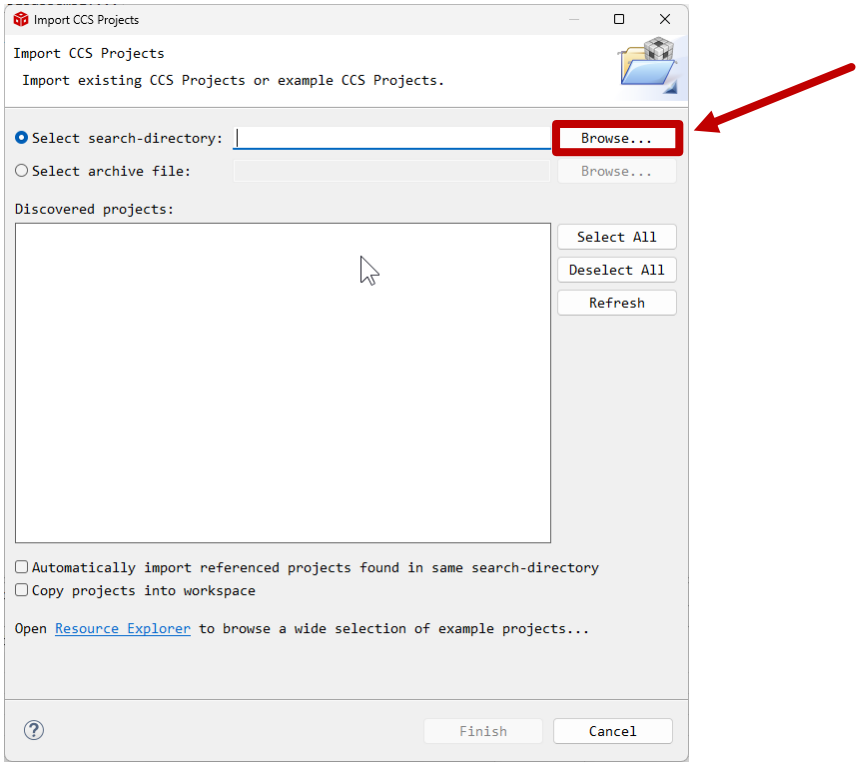


4.2 실습 따라 하기 (Build Level 1)

-------- 이어지는 과정은 CCS Edit 화면구성(Perspective)에서 진행됩니다. ---------

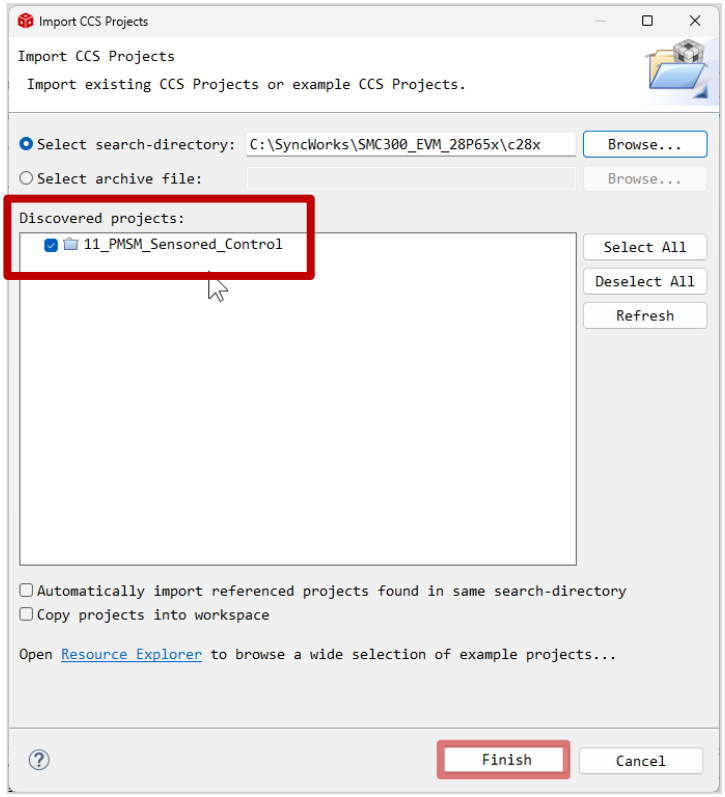
8. ‘PMSM_Sensored_Control’ 예제 프로젝트를 불러들입니다. (Open)

CCS 상단메뉴 Project → Import CCS Projects… 를 클릭합니다. 아래의 그림과 같은 창이 나타나면 ‘Select search-directory: ‘ 란 옆의 ‘Browse…’ 버튼을 클릭합니다.



디렉터리 탐색 창이 나타나면, Workspace 경로인 C:\SyncWorks\SMC300_EVM_28P65x\c28x 를 선택하고 ‘확인’ 버튼을 클릭합니다.

아래 그림과 같이 'Discovered project: ' 란의 프로젝트 목록에서 'PMSM_Sensored_Control' 이 선택되었음을 확인(프로젝트 이름 좌측의 체크박스)한 후, 'Finish' 버튼을 클릭합니다.



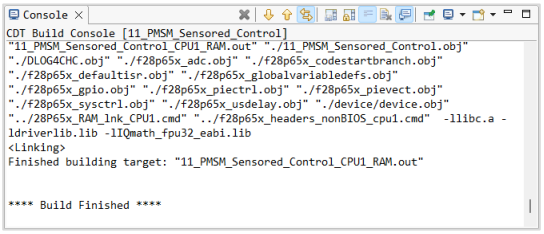
9. 헤더 파일의 BUILDLEVEL 설정을 확인 합니다.

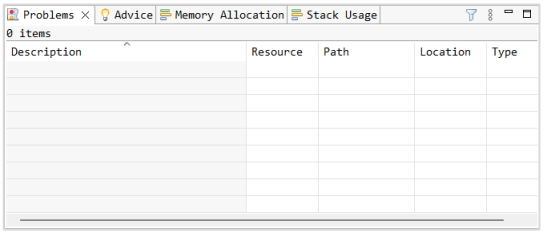
CCS 화면 좌측의 Project Explorer 창에서 '11_PMSM_Sensored_Control.h' 파일을 찾아 마우스 좌측 버튼으로 더블클릭 합니다. 화면 중앙에 에디터 창을 통해 헤더 파일이 열리면 BUILDLEVEL 항목이 'LEVEL1' 으로 정의되어 있는지 확인합니다.

```
39 #define CAL_ANG_TUNE    FALSE   // CalibrationAngle 값 재-교정 선택 옵션 (Build Level 3)
40
41 #define LEVEL1          1       // SVGEN_DQ 및 FC_PWM_DRV 모듈 테스트 (Open-loop)
42 #define LEVEL2          2       // 전류 검출 및 엔코더 신호 확인 (Open-loop)
43 #define LEVEL3          3       // 전류 제어(PI) 확인 / CalibrationAngle 재-교정 (옵션)
44 #define LEVEL4          4       // 속도제어 실행 (Closed-loop)
45 #define LEVEL5          5       // 위치제어 실행 (Closed-loop)
46
47 #define BUILDLEVEL      LEVEL1
```

10. 예제 프로젝트를 Build 합니다.

CCS 화면상단 단축 아이콘들 중  망치모양 아이콘을 클릭하거나, CCS 상단메뉴 Project → Build Project 를 클릭하여 예제 프로젝트를 Build 합니다. Build 과정이 정상적으로 완료되면 아래와 같이 콘솔(Console) 창에 11_PMSM_Sensored_Control_CPU1_RAM.out 파일이 생성되었음을 알리는 메시지가 나타나고 오류(Problems) 창에 아무런 메시지가 나타나지 않습니다.





11. CCS 화면구성을 CCS Debug로 전환합니다.

이제 Build 한 예제를 칩에 Load 하고 실행시켜보기 위해 CCS 화면구성을 CCS Debug로 전환합니다. CCS 우측 상단의 'CCS Debug' 버튼을 클릭해서 화면 구성을 전환하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

12. 예제의 Output 파일을 프로세서에 Load 합니다.

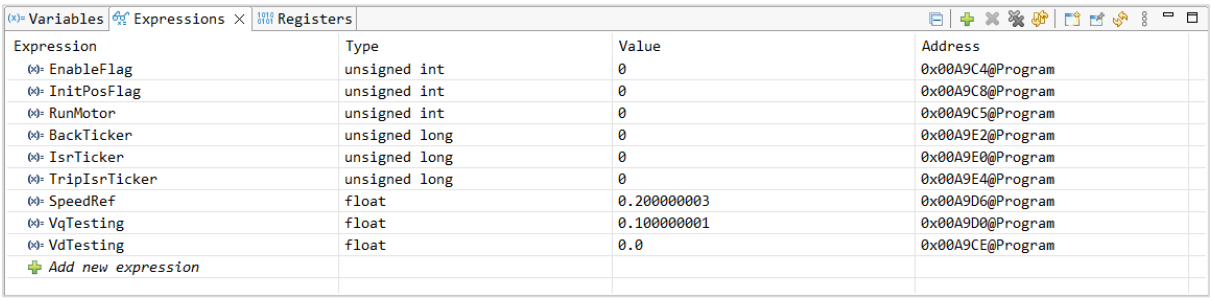
CCS 상단메뉴 Run → Load → Load Program… 을 선택합니다. Load Program 창이 나타나면 창 우측의 'Browse project…' 버튼을 클릭하고 Select a program 창에서 앞서의 과정에서 생성한 '11_PMSM_Sensored_Control_CPU1_RAM.out' 파일을 선택한 후 'OK' 버튼을 클릭합니다. Select a program 창이 사라지면 Load Program 창의 'OK' 버튼을 클릭해서 Output 파일을 칩에 Load 합니다.

13. Expressions 탭에 관찰 및 조작할 변수들을 등록합니다.

예제를 실행하면서 관찰 및 조작할 변수, 레지스터들을 Expressions 탭에 등록합니다. 이렇게 등록해둔 변수 및 레지스터들은 예제 실행 중 실시간으로 관찰 및 변경이 가능합니다. CCS 화면

우측 상단의 Expressions 탭을 선택한 후 해당 탭의 빈 공간을 마우스 우측 버튼으로 클릭합니다. 단축메뉴가 나타나면 'Import…' 항목을 선택하고 탐색기 창에서 아래 경로의 파일을 선택합니다. (해당 파일은 Build Level 1에서 관찰 및 조작할 변수들을 미리 저장해둔 것 입니다.) 그 외에 살펴보고 싶은 변수나 레지스터들이 있다면 추가로 등록하실 수 있습니다.

C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_PMSM_Sensored_Control\
Variables_BuildLevel1.txt


Variables | Expressions | Registers

Expression	Type	Value	Address
EnableFlag	unsigned int	0	0x00A9C4@Program
InitPosFlag	unsigned int	0	0x00A9C8@Program
RunMotor	unsigned int	0	0x00A9C5@Program
BackTicker	unsigned long	0	0x00A9E2@Program
IsrTicker	unsigned long	0	0x00A9E0@Program
TripIsrTicker	unsigned long	0	0x00A9E4@Program
SpeedRef	float	0.200000003	0x00A9D6@Program
VqTesting	float	0.100000001	0x00A9D0@Program
VdTesting	float	0.0	0x00A9CE@Program
Add new expression			

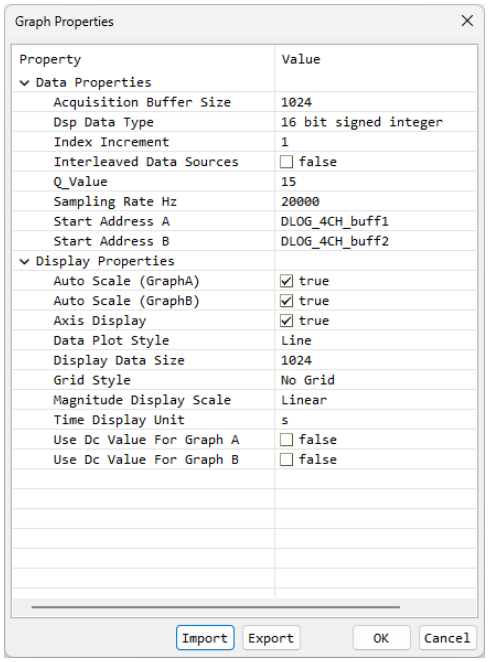
14. Graph 창을 생성합니다.

TI 디지털 모터제어 라이브러리에 포함된 DATA_LOG 모듈이 예제 실행 중 램 영역에 수집한 정보들을 그래프로 관찰하기 위해 아래의 과정을 진행합니다.

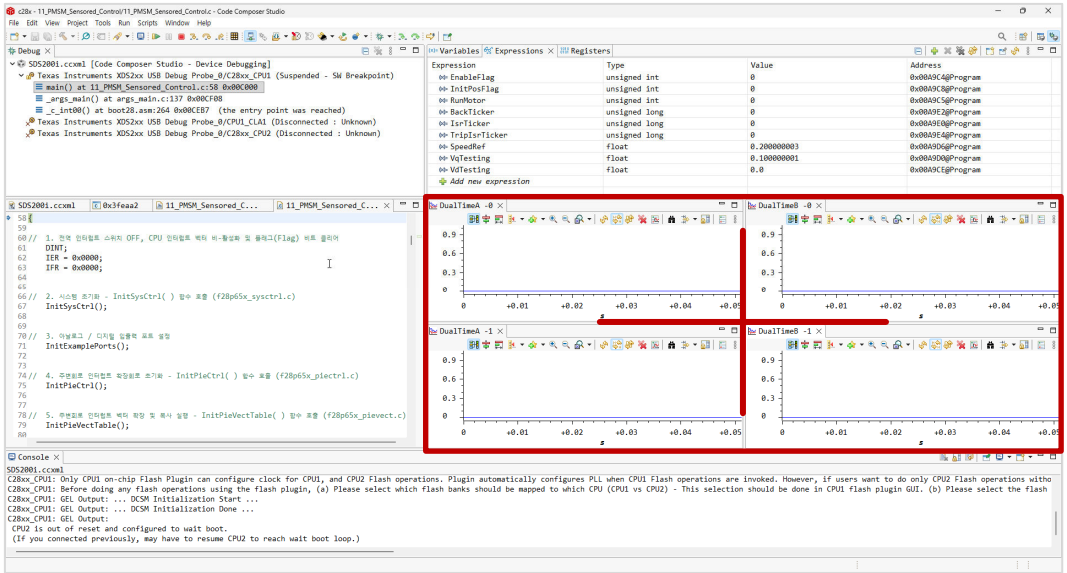
CCS 상단메뉴 Tools → Graph → Dual Time 을 선택하고, 그래프 설정 창 하단의 'Import' 버튼을 클릭합니다. 파일 탐색 창이 나타나면 아래 경로의 파일을 선택합니다.

C:\SyncWorks\SMC300_EVM_28P65x\c28x\11_PMSM_Sensored_Control\
GraphProperties1.graphProp

위의 파일은 데이터 관찰을 위해 생성할 그래프 창의 각종 설정을 미리 저장해둔 것 입니다. 불러들인 설정이 다음 이미지와 같다면 창 하단의 'OK' 버튼을 클릭해서 그래프 창을 생성합니다.



앞서의 과정을 참고하여, 동일한 경로의 GraphProperties2.graphProp 파일도 Import 해서 추가로 그래프 창을 생성해줍니다. 과정이 모두 완료되면 CCS 화면에 총 4개의 그래프 창이 나타납니다. 생성된 그래프 창은 마우스 왼쪽 버튼으로 창 이름 탭 부분을 클릭 및 드래그해서 원하는 위치로 정렬할 수 있습니다. (아래는 화면 우측 중앙으로 4개의 그래프 창을 정렬한 예입니다.)



15. 프로세서를 Reset 합니다.

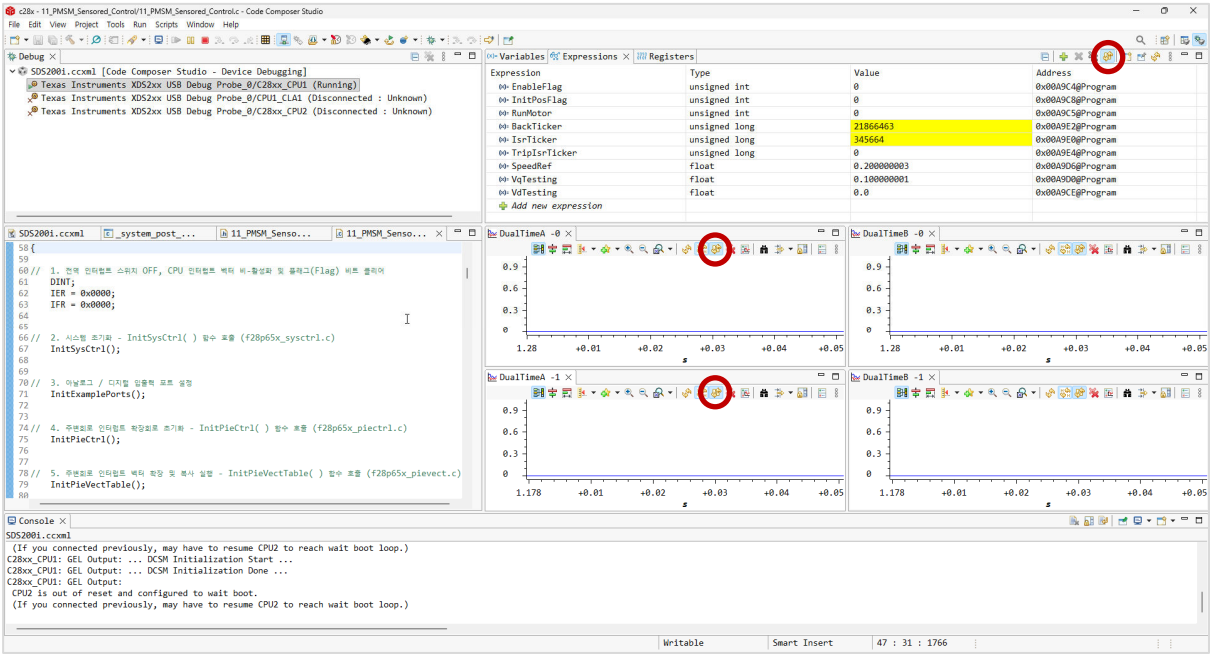
CCS 상단메뉴 Run → Reset → Reset CPU 를 실행합니다. 이어서 Run → Restart 메뉴를 실행합니다. 본 과정을 통해 예제를 실행하기 전 칩의 모든 레지스터들을 깨끗하게 초기 값으로 환원시킬 수 있습니다.

16. 예제 프로그램을 실행합니다.

CCS 상단메뉴 Run → Resume 을 선택하거나, 화면 좌측 상단 Debug 창 상단의 초록색 삼각형 아이콘  을 클릭하여 예제 프로그램을 실행합니다.

17. 앞서 생성한 Graph 창과 Expressions 탭의 실시간 갱신 기능을 켭니다.

Expressions 탭과 Graph 창 우측 상단의  아이콘을 클릭하여, Continuous Refresh 옵션을 켭니다. 이 기능을 통해 예제 실행 중 Expressions 탭에 등록된 변수들과 그래프 창의 내용이 실시간으로 갱신됩니다.



18. SMC300 개발보드의 DC-LINK [A] 전원 스위치를 Ctrl. 위치로 설정합니다.

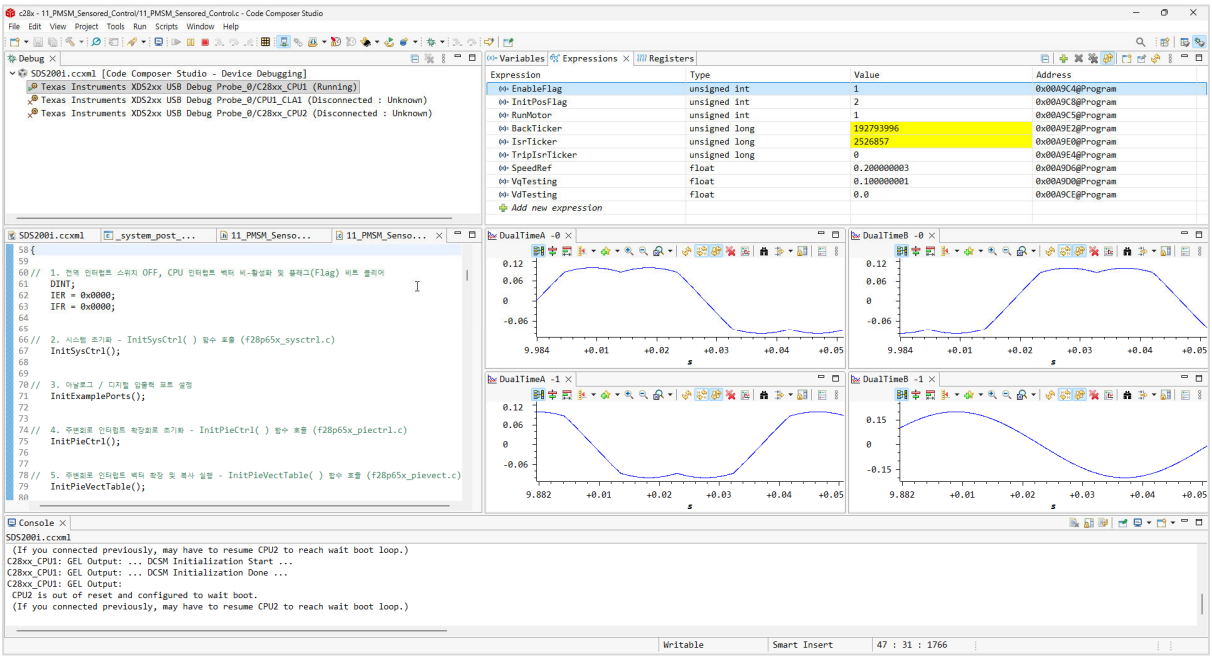
본 예제는 SMC300 개발보드의 모터제어 블록 A를 사용합니다. SMC300 개발보드의 모터제어 블록 A의 DC-LINK 전원 제어용 스위치(SW1400)를 'Ctrl.' 위치로 변경하여 모터제어에 사용될 전원의 ON/OFF를 예제 프로그램에서 제어할 수 있도록 합니다. **(SMC300 EVM V100 버전에 한 함 / SMC300 EVM V110 버전의 경우 물리 스위치가 제거되었으며, eFuse 스위치를 통해 전자제어 방식으로 DC-LINK 전원을 제어합니다.)**

19. SMC300 개발보드의 ADC VREFHI 선택 스위치를 3.3V 위치로 설정합니다.

모터제어 블록 A의 ADC VREFHI 선택 스위치 2개를 모두 '3.3V' 위치로 변경하여, 모터제어 블록의 아날로그 신호 가공 회로가 TMS320F28P65x 모듈의 기본 ADC 참조전압 설정과 일치하는 신호를 생성할 수 있도록 합니다. (Internal 1.65V Reference Mode / 3.3V Reference Range)

20. Expressions 탭에 등록되어 있는 'EnableFlag' 변수를 '0' 에서 '1'로 변경합니다.

앞서 Expressions 탭에 등록해둔 주요 변수들 중 'EnableFlag' 변수의 값을 '1'로 변경합니다. 예제 프로그램은 해당 Flag가 1로 변경되면 모터제어에 필요한 알고리즘들이 실행되도록 구성되어 있습니다.



21. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 1은 모터 축의 부하와 관계없이 일정한 Duty-ratio의 PWM을 이용하는 Open-Loop 단계이며, Space-Vector-PWM 생성을 담당하는 SVGEN_DQ, FC_PWM_DRV 모듈의 정상동작 여부를 확인하는 단계 입니다.

[조작 대상 변수들 : SpeedRef, VqTesting]

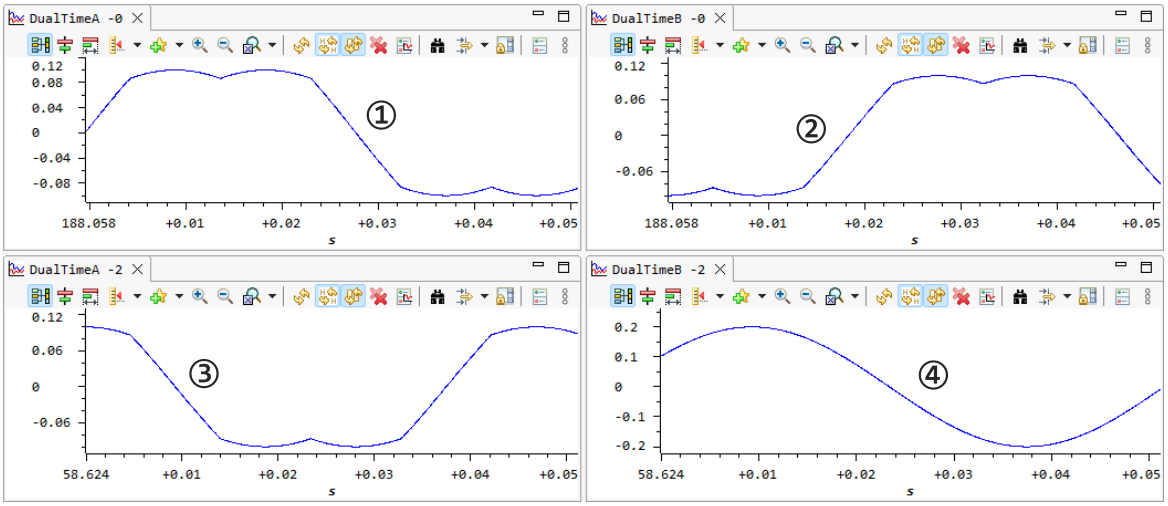
SpeedRef 변수를 -0.5 ~ +0.5 까지의 비율 범위에서 조작해 보실 수 있습니다. 이 변수는 사용자 속도지령에 해당되는 변수로 모터의 Base RPM (MAX. RPM) 기준 속도지령을 비율로 환산한 값 입니다. (0.0 = 0 RPM, 1.0 = Base RPM) SpeedRef 변수의 값이 변경되면 실제 Encoder 대신 모터 회전자의 위치 정보를 프로그램 적으로 시뮬레이션 하는 Ramp Generator(rg1) 모듈의 출력 신호 주파수가 변경되어 모터의 회전속도가 달라지게 됩니다. 후에 PI 제어를 적용하여 Closed-loop 구동 시에는 속도 지령에 따라 모터가 회전속도를 추종하는데 필요한 만큼의 에너지(PWM Duty-ratio)만을 적절히 제어하지만, 본 단계는 Open-Loop로 VqTesting 변수에 의해 결정되는 고정적인 Q축 전류 지령을 이용하기 때문에 SpeedRef 변수를 -0.5 ~ +0.5 범위 안에서만 조정해 보시기 바랍니다. VqTesting 변수는 Q축 전류 지령에 해당되는 비율(0.0 ~ 1.0) 변수 입니다. Level 1 실행 시 0.0 ~ 0.2 사이에서만 조작해보시길 권해드립니다. 해당 변수의 값을 크게 설정할 경우 SpeedRef 변수의 의해 결정된 모터의 회전속도를 유지하는데 필요한 것보다 과다한 에너지가 모터에 공급되게 되어 모터 드라이버 IC나 모터에 무리가 될 수 있습니다. (주의! : VqTesting 변수의 값이 크게 설정될 경우 드라이버 IC와 모터의 발열이 심해지며 고장의 원인이 될 수 있습니다.)

[주요 관찰 대상 변수들 : 없음]

Level 1 단계는 모터 구동에 사용되는 Space-Vector-PWM 생성과 관련된 S/W 모듈의 정상동작 여부를 확인하는 Open-Loop 단계로 Feedback 되는 정보가 없기에 별도로 관찰할 변수는 없습니다. 임의의 지령 값(속도 및 Q축 전류)에 의해 모터가 정상적으로 회전하는지 만 확인하시면 됩니다.

Build Level 1에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
225
226     //  Connect inputs of the DATALOG module
227     DlogCh1 = (int16)_IQ15(svgen_dq1.Ta);
228     DlogCh2 = (int16)_IQ15(svgen_dq1.Tb);
229     DlogCh3 = (int16)_IQ15(svgen_dq1.Tc);
230     DlogCh4 = (int16)_IQ15(svgen_dq1.Ta-svgen_dq1.Tb);
231
232 #endif  // BUILDLEVEL == LEVEL1
233
```



22. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A – DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.

4.3 실습 따라 하기 (Build Level 2)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 2(헤더 파일의 정의 구문 수정)로 설정하여 실행합니다.

Build Level 수정 : 11_PMSM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL2)

```
39 #define CAL_ANG_TUNE    FALSE   // CalibrationAngle 값 재-교정 선택 옵션 (Build Level 3)
40
41 #define LEVEL1          1       // SVGEN_DQ 및 FC_PWM_DRV 모듈 테스트 (Open-loop)
42 #define LEVEL2          2       // 전류 검출 및 엔코더 신호 확인 (Open-loop)
43 #define LEVEL3          3       // 전류 제어(PI) 확인 / CalibrationAngle 재-교정 (옵션)
44 #define LEVEL4          4       // 속도제어 실행 (Closed-loop)
45 #define LEVEL5          5       // 위치제어 실행 (Closed-loop)
46
47 #define BUILDLEVEL      LEVEL2
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel2.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고, 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1' 로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

23. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 2는 Level 1과 동일한 Open-Loop 구동 단계로, Level 1에서 사용된 S/W 모듈들에 추가로 On-Chip ADC를 구동하여 모터의 상-전류를 얻어내는 S/W 모듈이 사용됩니다. 모터가 회전함에 따라, 모터의 U상과 V상 전류가 올바르게 검출되는지 확인해볼 수 있습니다. 이와 더불어 Encoder가 주는 신호를 해석한 실제 모터 회전자의 위치정보를 확인해봅니다. (그래프 창을 통해 Encoder의 출력신호를 해석한 모터 회전자의 위치정보와 ADC가 검출해낸 U, V상 전류를 관찰해보세요.)

[조작 대상 변수들 : SpeedRef, VqTesting]

SpeedRef 변수를 -0.5 ~ +0.5 까지의 비율 범위에서 조작해보실 수 있습니다. 이 변수는 사용자 속도지령에 해당되는 변수로 모터의 Base RPM (MAX. RPM) 기준 속도지령을 비율로 환산한 값 입니다. (0.0 = 0 RPM, 1.0 = Base RPM) SpeedRef 변수의 값이 변경되면 실제 Encoder 대신 모터 회전자의 위치 정보를 프로그램 적으로 시뮬레이션 하는 Ramp Generator(rg1) 모듈의 출력 신호 주파수가 변경되어 모터의 회전속도가 달라지게 됩니다. 후에 PI 제어를 적용하여 Closed-loop 구동 시에는 속도 지령에 따라 모터가 회전속도를 추종하는데 필요한 만큼의 에너지(PWM Duty-ratio)만을 적절히 제어하지만, 본 단계는 Open-Loop로 VqTesting 변수에 의해 결정되는 고정적인 Q축 전류 지령을 이용하기 때문에 SpeedRef 변수를 -0.5 ~ +0.5 범위 안에서만 조정해보시기 바랍니다. VqTesting 변수는 Q축 전류 지령에 해당되는 비율(0.0 ~ 1.0) 변수 입니다. Level 2 실행 시 0.0 ~ 0.2 사이에서만 조작해보시길 권해드립니다. 해당 변수의 값을 크게 설정할 경우 SpeedRef 변수의 의해 결정된 모터의 회전속도를 유지하는데 필요한 것보다 과다한 에너지가 모터에 공급되게 되어 모터 드라이버 IC나 모터에 무리가 될 수 있습니다. (주의! : VqTesting 변수의 값이 크게 설정될 경우 드라이버 IC와 모터의 발열이 심해지며 고장의 원인이 될 수 있습니다.)

[주요 관찰 대상 변수들 : ilg2_vdc1 구조체, qep1 구조체]

Level 2는 Open-Loop 모터 구동과 함께 On-Chip ADC를 통한 상-전류 검출, EQEP를 이용한 Encoder 신호 해석이 정상인지 확인하는 단계입니다. 임의의 지령 값(속도 및 Q축 전류)에 의해 모터가 회전하면서 On-Chip ADC에 의해 U, V 상 전류가 정상적으로 검출되는지, Encoder로부터의 신호를 EQEP 모듈이 받아 회전자의 위치정보를 잘 표시하고 있는지 확인해보세요. (그래프 창 참조)

Build Level 2에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

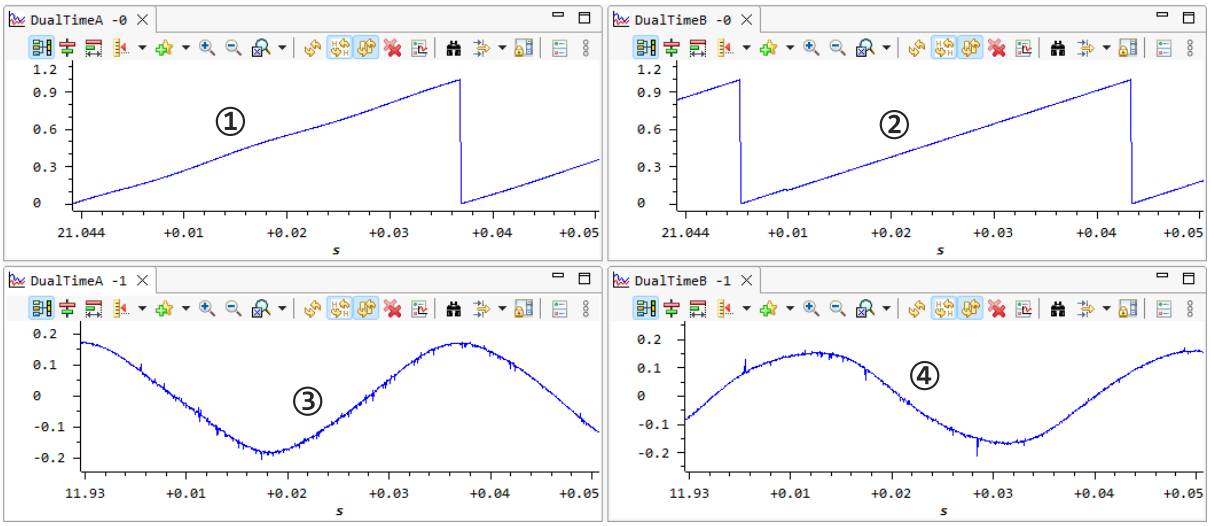
```
280     //  Connect inputs of the DATALOG module
281     DlogCh1 = (int16)_IQ15(qep1.ElecTheta);
282     DlogCh2 = (int16)_IQ15(rg1.Out);
283     DlogCh3 = (int16)_IQ15(clarke1.As);
284     DlogCh4 = (int16)_IQ15(clarke1.Bs);
```

qep1.ElecTheta : Encoder 출력신호를 기초로 해석한 위치정보

rg1.Out : Open-Loop 모터 구동을 위해 임의로 생성한 위치정보 시뮬레이션 값

clarke1.As : A상 Current Feedback

clarke1.Bs : B상 Current Feedback



24. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A – DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.

4.4 실습 따라 하기 (Build Level 3)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 3(헤더 파일의 정의 구문 수정)으로 설정하여 실행합니다.

Build Level 수정 : 11_PMSM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL3)

```
39 #define CAL_ANG_TUNE    FALSE   // CalibrationAngle 값 재-교정 선택 옵션 (Build Level 3)
40
41 #define LEVEL1          1       // SVGEN_DQ 및 FC_PWM_DRV 모듈 테스트 (Open-loop)
42 #define LEVEL2          2       // 전류 검출 및 엔코더 신호 확인 (Open-loop)
43 #define LEVEL3          3       // 전류 제어(PI) 확인 / CalibrationAngle 재-교정 (옵션)
44 #define LEVEL4          4       // 속도제어 실행 (Closed-loop)
45 #define LEVEL5          5       // 위치제어 실행 (Closed-loop)
46
47 #define BUILDLEVEL      LEVEL3
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel3.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고, 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1' 로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

25. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 3은 Level 2에서 확인한 두 상의 Current Feedback에 PI 제어 모듈을 적용하여 Current Closed-Loop를 확인하는 단계입니다. 모터의 구동은 Level 1, 2와 마찬가지로 임의의 위치정보를 이용한 Open-Loop로 이뤄지지만, 초기에 Encoder Index 신호 위치를 찾은 후 모터가 회전하도록 하거나 임의의 시작 위치와 실제 Index 신호 위치와의 차이를 보정한 후 모터가 회전하도록 프로그램이 구성되어 있습니다. 헤더파일의 CAL_ANG_TUNE 설정이 FALSE 일 경우 PrePosInitialize( ) 함수가 실행되어 Encoder의 Index 펄스 위치를 찾은 후 모터를 구동하며, CAL_ANG_TUNE 설정이 TRUE 일 경우 TI가 공개해둔 PMSM 예제와 동일하게 Angle

값을 강제로 '0'으로 설정하여 임의의 시작위치에 고정하고 실제 Index 펄스 위치와의 오차를 qep1.CalibratedAngle 변수에 적용하는 과정이 진행됩니다.

[CAL_ANG_TUNE 항목을 'FALSE'로 정의한 경우]

예제를 실행하고 Expressions 탭의 'EnableFlag' 변수를 '1'로 변경하면, PrePosInitialize( ) 함수가 실행되면서 모터가 Open-Loop 구동을 통해 천천히 회전하여 Encoder의 Index 펄스 위치를 찾습니다. Index 펄스가 감지되고 나면, Index 펄스 위치에서 잠시 대기한 후 본 코드가 실행되면서 모터가 SpeedRef 변수에 의해 설정된 속도로 회전(Speed Open-Loop, Current Closed-Loop)하게 됩니다.

[CAL_ANG_TUNE 항목을 'TRUE'로 정의한 경우]

예제를 실행하고 Expressions 탭의 'EnableFlag' 변수를 '1'로 변경하면, 모터가 임의의 초기위치(rg1.Angle = 0)에서 정지한 상태로 사용자의 다음 입력을 기다립니다.
이때 Expressions 탭에 등록된 LockFlag 변수를 '0' 에서 '1' 로 변경하면 본 코드가 실행되면서 모터가 사용자가 설정한 SpeedRef 변수에 의해 해당 속도로 회전(Speed Open-Loop, Current Closed-Loop)하게 되며, qep1.CalibratedAngle 변수에 임의의 초기위치와 엔코더의 Index 펄스와의 옵셋 값이 기록됩니다. 이렇게 찾아낸 옵셋 값을 메인 함수의 초기화 코드에 반영하거나 매 프로그램 실행 시 마다 옵셋을 다시 찾도록 할 수 있습니다. 이 옵셋 값은 서로 다른 초기 위치에 따라 매번 달라질 수 있으며 수식으로 표현하면 아래와 같습니다.

Calibration Angle = Offset Angle ± (n * Line Encoder)

모터가 정상 회전한 후 옵셋 값을 정해진 속도 조건 하에서 최소한의 전류를 소비하도록 미세 조정할 수 있습니다.

[Level 3 조작 대상 변수들 : SpeedRef, IqRef]

SpeedRef 변수를 -0.5 ~ +0.5 까지의 비율 범위에서 조작해보실 수 있습니다. 이 변수는 사용자 속도지령에 해당되는 변수로 모터의 Base RPM (MAX. RPM) 기준 속도지령을 비율로 환산한 값 입니다. (0.0 = 0 RPM, 1.0 = Base RPM) SpeedRef 변수의 값이 변경되면 실제 Encoder 대신 모터 회전자의 위치 정보를 시뮬레이션 하는 Ramp Generator(rg1) 모듈의 출력 신호 주파수가 변경되어 모터의 회전속도가 달라지게 됩니다. 후에 PI 제어를 적용하여 Speed Closed-loop 구동 시(Level 4)에는 속도 지령에 따라 모터가 회전속도를 추종하는데 필요한

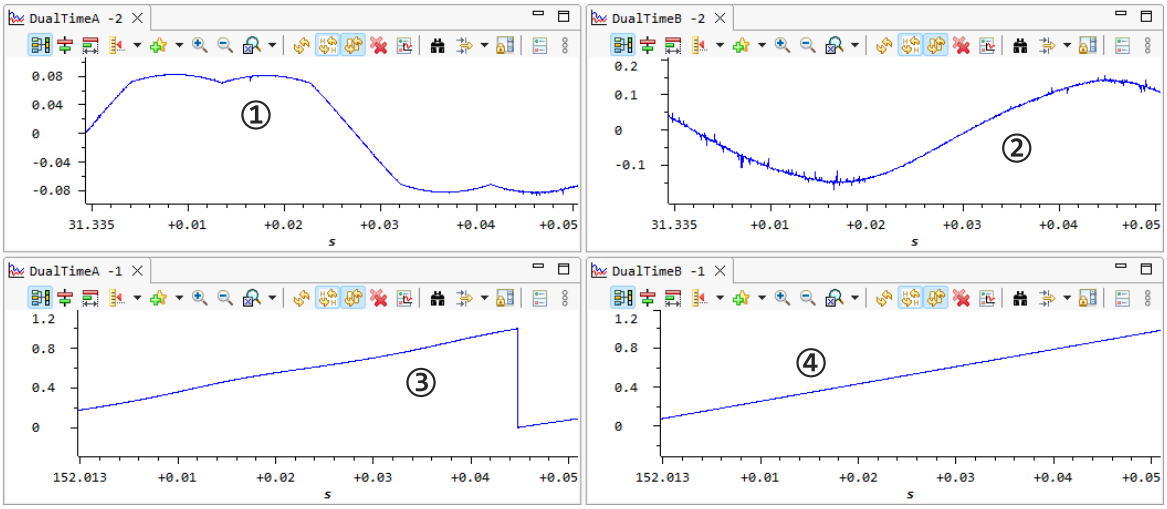
만큼의 에너지(PWM Duty-ratio)만을 적절히 제어하지만, 본 단계는 임의의 위치 정보에 의한 Open-Loop 구동 단계로 IqRef 변수와 그에 연결된 PI 제어기에 의해 일정한 크기의 전류가 모터에 공급될 뿐 속도에 따라 가변되지 않으므로 SpeedRef 변수를 -0.5 ~ +0.5 범위 안에서만 조정해보시기 바랍니다. IqRef 변수는 Q축 전류 지령에 해당되는 비율(0.0 ~ 1.0) 변수 입니다. Level 3 실행 시 0.0 ~ 0.2 사이에서만 조작해보시길 권해드립니다. 해당 변수의 값을 크게 설정할 경우 SpeedRef 변수에 의해 결정된 모터의 회전속도를 유지하는데 필요한 것보다 과다한 에너지가 모터에 공급되게 되어 모터 드라이버 IC나 모터에 무리가 될 수 있습니다. (주의! : IqRef 변수의 값이 크게 설정될 경우 드라이버 IC와 모터의 발열이 심해지며 장시간 구동 시 고장의 원인이 될 수 있습니다.)

[주요 관찰 대상 변수들 : pid1_iq 구조체, speed1 구조체]

Level 3은 Open-Loop 모터 구동과 함께 On-Chip ADC를 통해 검출한 상-전류를 기초로 산출한 Q축 PI 제어기의 동작을 확인해보는 단계 입니다. 아울러 Encoder가 검출한 위치정보를 바탕으로 현재 모터의 회전속도를 계산하는 speed1 모듈의 동작도 살펴보시기 바랍니다. speed1.Speed 변수는 초기화 과정에서 정의한 Base RPM 대비 현재 모터의 속도를 -1.0 ~ +1.0 사이의 비율로 표현하고 이를 RPM으로 환산한 값이 speed1.SpeedRpm 변수에 저장됩니다.

Build Level 3에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
441     //  Connect inputs of the DATALOG module
442     DlogCh1 = (int16)_IQ15(svgen_dq1.Ta);
443     DlogCh2 = (int16)_IQ15(clarke1.As);
444     DlogCh3 = (int16)_IQ15(qep1.ElecTheta);
445     DlogCh4 = (int16)_IQ15(rg1.Out);
```



26. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A – DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.

4.5 실습 따라 하기 (Build Level 4)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 4(헤더 파일의 정의 구문 수정)로 설정하여 실행합니다.

Build Level 수정 : 11_PMSM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL4)

```
39 #define CAL_ANG_TUNE    FALSE   // CalibrationAngle 값 재-교정 선택 옵션 (Build Level 3)
40
41 #define LEVEL1          1       // SVGEN_DQ 및 FC_PWM_DRV 모듈 테스트 (Open-loop)
42 #define LEVEL2          2       // 전류 검출 및 엔코더 신호 확인 (Open-loop)
43 #define LEVEL3          3       // 전류 제어(PI) 확인 / CalibrationAngle 재-교정 (옵션)
44 #define LEVEL4          4       // 속도제어 실행 (Closed-loop)
45 #define LEVEL5          5       // 위치제어 실행 (Closed-loop)
46
47 #define BUILDLEVEL      LEVEL4
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel4.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고, 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1' 로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

27. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 4는 Q축 PI 제어기 외곽에 Speed PI 제어기를 하나 더 추가하여, 사용자의 속도지령을 모터가 추종하도록 하는 Speed Closed-Loop 단계 입니다. Speed PI 제어기의 출력이 Q축 PI 제어기의 Reference가 되어 부하에 의해 모터의 회전속도가 달라질 경우 모터에 전달하는 에너지를 증가시켜 설정된 속도 값을 추종하게 됩니다. 예제 프로그램을 실행한 후 Expressions 탭에 등록된 EnableFlag 변수를 '1'로 변경하면 PrePosInitialize( ) 함수가 실행되며 Open-Loop 구동을 통해 Encoder Index 펄스의 위치를 찾습니다. 그 후 정상적으로 모터가 회전하고 나면, 모터의 회전축을 손으로 살짝 잡아 부하를 가해봅니다. 모터에 공급되는

에너지가 늘어나면서 설정된 속도를 고수하려고 하는 것을 확인하실 수 있습니다.

[Level 4 조작 대상 변수들 : SpeedRef]

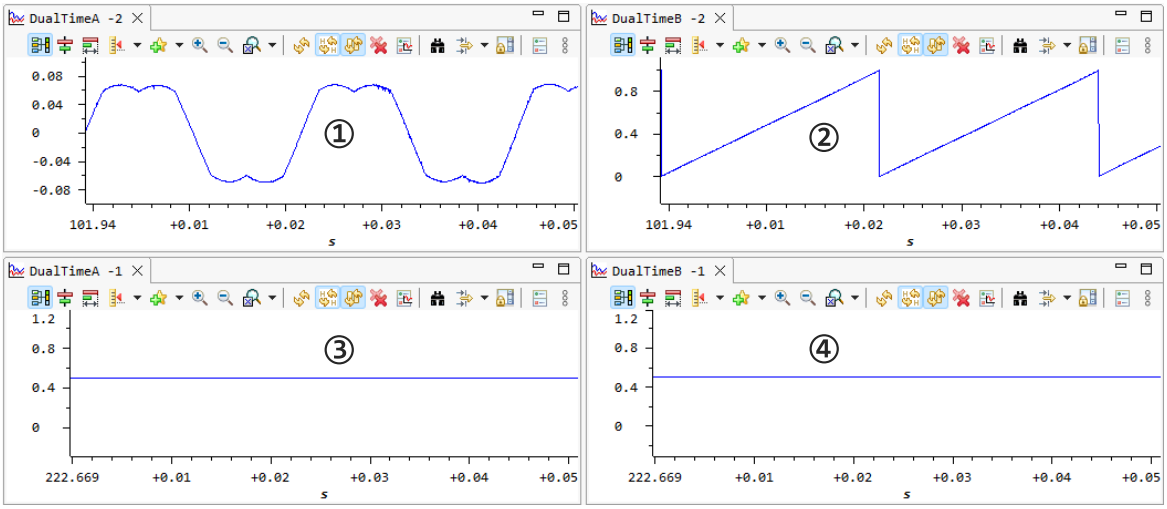
SpeedRef 변수를 -0.95 ~ +0.95 까지의 비율 범위에서 조작해보실 수 있습니다. 이 변수는 사용자 속도지령에 해당되는 변수로 모터의 Base RPM (MAX. RPM) 기준 속도지령을 비율로 환산한 값 입니다. (0.0 = 0 RPM, 1.0 = Base RPM, -1.0 = - Base RPM) SpeedRef 변수의 값이 변경되면 qep1 모듈과 speed1 모듈에 의해 산출된 모터의 실 회전속도를 Feedback으로 하여 실행되는 Speed PI 제어 모듈 출력이 변화하여 이를 Reference로 하는 Q축 PI 제어 모듈의 출력이 모터에 전달하는 에너지를 적절히 조절하게 됩니다. 결과적으로 부하의 변동에도 사용자가 설정한 속도 지령 값(SpeedRef)을 추종하도록 하는 제어가 이뤄지게 됩니다. (Speed Closed-Loop)

[주요 관찰 대상 변수들 : pid1_spd 구조체, pid1_iq 구조체, speed1 구조체]

Level 4는 Speed Closed-Loop 단계 입니다. qep1 모듈이 Encoder 신호를 통해 알아낸 모터 회전축의 위치 정보로 speed1 모듈이 현재 모터의 회전속도를 산출해냅니다. 이렇게 알아낸 속도와 사용자의 속도지령(SpeedRef)을 기초로 pid1_spd 모듈이 Output을 계산하고 이는 pid1_iq 모듈의 Reference로 연결되어 모터에 전달되는 에너지를 제어합니다. 속도제어 루프를 구성하는 주요 모듈들의 하위 변수들을 관찰해보세요,

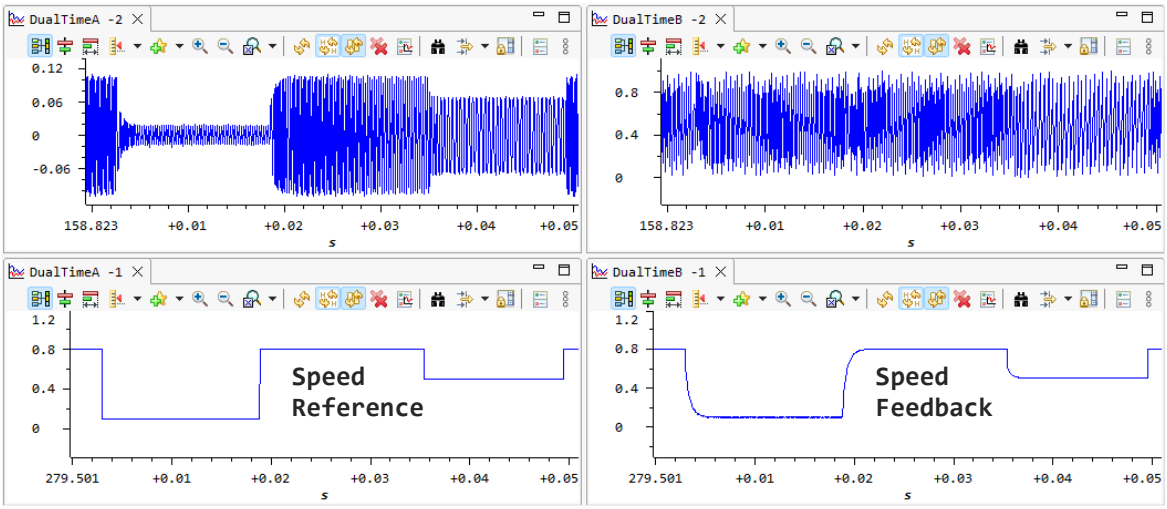
Build Level 4에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
519
520     //  Connect inputs of the DATALOG module
521     DlogCh1 = (int16)_IQ15(svgen_dq1.Ta);
522     DlogCh2 = (int16)_IQ15(speed1.ElecTheta);
523     DlogCh3 = (int16)_IQ15(pid1_spd.Ref);
524     DlogCh4 = (int16)_IQ15(pid1_spd.Fdb);
525
```



[속도 변화 이력 관찰 팁]

Level 4 에서는 CCS 그래프 창 3번, 4번을 통해 사용자 속도 지령 변화나 모터 축의 부하 변동에 따른 속도 PI 제어의 결과를 관찰할 수 있습니다. 기본적으로는 DATA_LOG 모듈이 매 제어주기마다 각 채널에 연결된 데이터를 누적하도록 되어 있기 때문에 긴 시간 범위에서의 변화 이력을 관찰하기 어렵습니다. Expressions 탭에 등록된 dlog.prescalar 변수를 '1'에서 '500'으로 변경해보세요. DATA_LOG 모듈이 제어 주기 500회 마다 1번씩만 데이터를 누적하여 보다 긴 시간 범위에서 Speed PI 제어 모듈의 Reference와 Feedback의 변화 이력을 관찰하실 수 있습니다.



28. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A – DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.

4.6 실습 따라 하기 (Build Level 5)

예제 프로젝트를 Build 하고, 실행하기까지의 과정은 Build Level 1 실습 때와 동일합니다. 앞서 설명된 내용과 아래를 참고하여 예제 프로젝트를 Build Level 5(헤더 파일의 정의 구문 수정)로 설정하여 실행합니다.

Build Level 수정 : 11_PMSM_Sensored_Control.h 파일의 'BUILDLEVEL' 값 변경 (LEVEL5)

```
39 #define CAL_ANG_TUNE    FALSE   // CalibrationAngle 값 재-교정 선택 옵션 (Build Level 3)
40
41 #define LEVEL1          1       // SVGEN_DQ 및 FC_PWM_DRV 모듈 테스트 (Open-loop)
42 #define LEVEL2          2       // 전류 검출 및 엔코더 신호 확인 (Open-loop)
43 #define LEVEL3          3       // 전류 제어(PI) 확인 / CalibrationAngle 재-교정 (옵션)
44 #define LEVEL4          4       // 속도제어 실행 (Closed-loop)
45 #define LEVEL5          5       // 위치제어 실행 (Closed-loop)
46
47 #define BUILDLEVEL      LEVEL5
```

Expressions 탭 변수등록용 파일 : Variables_BuildLevel5.txt

Graph 창 생성용 파일 1 : GraphProperties1.graphProp

Graph 창 생성용 파일 2 : GraphProperties2.graphProp

예제 프로그램을 Build 하고, 실행해서 Expressions 탭에 등록된 'EnableFlag'를 '1' 로 변경하세요.

-------- 이어지는 과정은 CCS Debug 화면구성(Perspective)에서 진행됩니다. ---------

-------- 예제 실행 명령(Run → Resume) 후의 과정 안내입니다. ---------

29. 예제 실행결과를 확인하고, 주요 변수들을 조작해봅니다.

Build Level 5는 Q축 PI 제어기 외곽에 Position PI 제어기를 하나 더 추가하여, 사용자의 위치지령을 모터가 추종하도록 하는 Position Closed-Loop 단계 입니다. Position PI 제어기의 출력이 Q축 PI 제어기의 Reference가 되어 부하에 의해 모터의 정지위치가 달라질 경우 모터에 전달하는 에너지를 증가시켜 설정된 지령 위치를 추종하게 됩니다. 예제 프로그램을 실행한 후 Expressions 탭에 등록된 EnableFlag 변수를 '1'로 변경하면 PrePosInitialize( ) 함수가 실행되며 Open-Loop 구동을 통해 Encoder Index 펄스의 위치를 찾습니다. 그 후 정상적으로 모터가 회전하고 나면, 모터의 회전축을 손으로 살짝 잡아 부하를 가해봅니다. 모터에 공급되는 에너지가 늘어나면서 설정된 지령 위치를 고수하려고 하는 것을 확인하실 수 있습니다.

[Level 5 조작 대상 변수들 : PositionRef]

PositionRef 변수를 0.05 ~ 0.95 까지의 비율 범위에서 조작해보실 수 있습니다. 이 변수는 사용자 위치지령에 해당되는 변수로 모터 회전 축 위치 18° ~ 342°를 비율적으로 환산한 값 입니다. (0.05 = 18°, 0.95 = 342°) PositionRef 변수의 값이 변경되면 qep1 모듈이 알아낸 모터의 실제 위치를 Feedback으로 하여 실행되는 Position PI 제어 모듈(pid1_pos) 출력이 변화하여 이를 Reference로 하는 Q축 PI 제어 모듈의 출력이 모터에 전달하는 에너지를 적절히 조절하게 됩니다. 결과적으로 부하의 변동에도 사용자가 설정한 정지위치 지령 값(PositionRef)을 추종하도록 하는 제어가 이뤄지게 됩니다. (Position Closed-Loop)

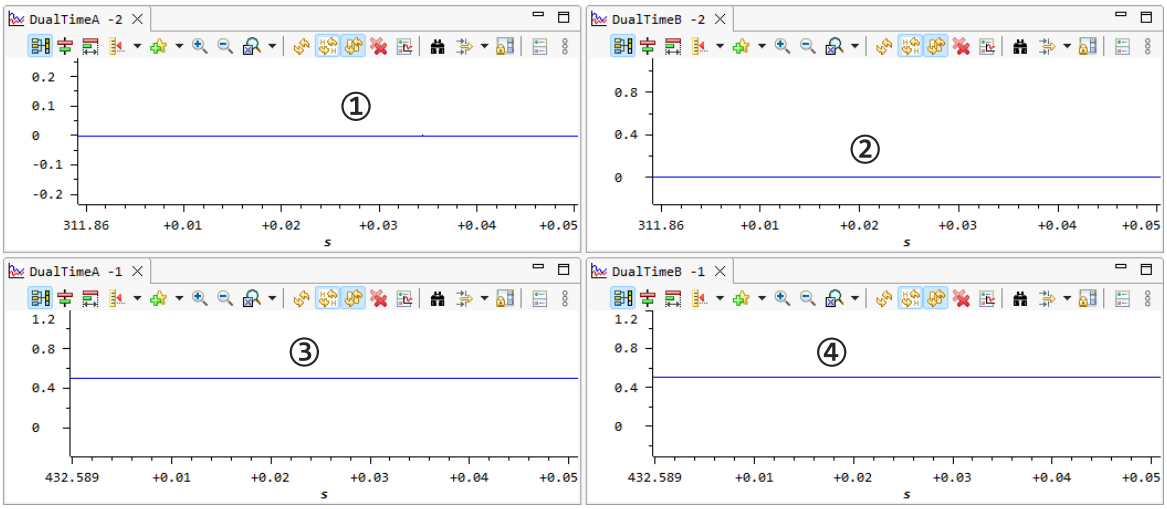
[주요 관찰 대상 변수들 : pid1_pos 구조체, pid1_iq 구조체, qep1 구조체]

Level 5는 Position Closed-Loop 단계 입니다. qep1 모듈이 Encoder 신호를 통해 알아낸 모터 회전축의 위치 정보와 사용자의 정지위치지령(PositionRef)을 기초로 pid1_pos 모듈이 Output을 계산하고 이는 pid1_iq 모듈의 Reference로 연결되어 모터에 전달되는 에너지를

제어합니다. 위치제어 루프를 구성하는 주요 모듈들의 하위 변수들을 관찰해보세요,

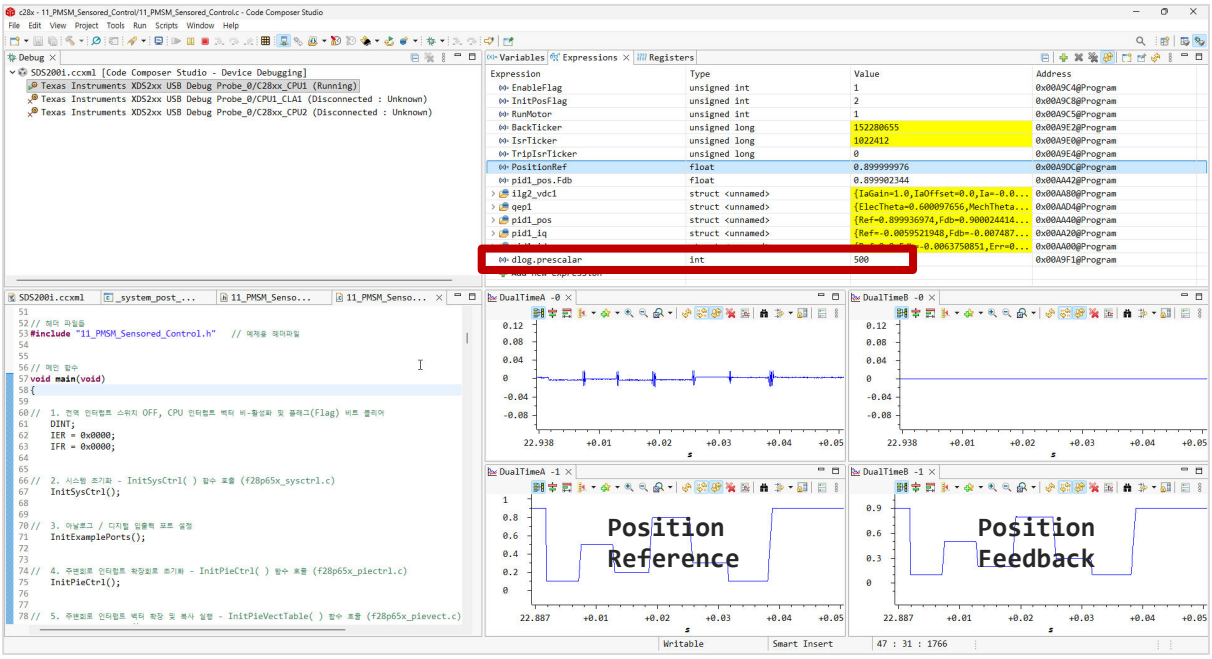
Build Level 5에서는 아래와 같은 그래프 창을 확인하실 수 있으며, 아래의 코드를 통해 DATA_LOG 모듈이 어떤 변수들의 변화이력을 누적하고 있는지 알 수 있습니다. (DATA_LOG 모듈의 1~4 채널 ➔ 그래프 창 1~4)

```
584     //  Connect inputs of the DATALOG module
585     DlogCh1 = (int16)_IQ15(svgen_dq1.Ta);
586     DlogCh2 = (int16)_IQ15(speed1.ElecTheta);
587     DlogCh3 = (int16)_IQ15(pid1_pos.Ref);
588     DlogCh4 = (int16)_IQ15(pid1_pos.Fdb);
```



[위치 변화 이력 관찰 팁]

Level 5 에서는 CCS 그래프 창 3번, 4번을 통해 사용자 위치 지령 변화나 모터 축의 부하 변동에 따른 위치 PI 제어의 결과를 관찰할 수 있습니다. 기본적으로는 DATA_LOG 모듈이 매 제어주기마다 각 채널에 연결된 데이터를 누적하도록 되어 있기 때문에 긴 시간 범위에서의 변화 이력을 관찰하기 어렵습니다. Expressions 탭에 등록된 dlog.prescalar 변수를 '1'에서 '500'으로 변경해보세요. DATA_LOG 모듈이 제어 주기 500회 마다 1번씩만 데이터를 누적하여 보다 긴 시간 범위에서 Position PI 제어 모듈의 Reference와 Feedback의 변화 이력을 관찰하실 수 있습니다.



30. 안전하게 예제 프로그램의 실행을 중지합니다.

아래의 순서에 따라 안전하게 예제의 실행을 중지합니다.

a). Expressions 탭에 등록된 'EnableFlag' 변수를 '0'으로 변경합니다. (변경 후 모터가 정지하는 것을 확인합니다.)

b). SMC300 개발보드의 모터제어 블록 A - DC-LINK [A] 전원 스위치(SW1400)를 'OFF' 위치로 변경하면 하드웨어적으로 모터 구동용 전원을 차단할 수 있습니다. (SMC300 EVM V100에 한 함)

c). CCS 상단메뉴 Run → Suspend 를 클릭하여 프로그램 실행을 중지합니다. (Halt Program)

d). CCS를 종료하고 SMC300 개발보드의 통합 전원 스위치(PSW1000)를 'OFF' 위치로 변경하여 보드에 공급된 전원을 차단합니다.

e). 전원 스위치를 OFF로 조작 후, 수 초간 기다렸다가 JTAG 에뮬레이터 및 프로세서 모듈을 보드와 분리합니다.